Custom Repository Implementations

Custom Repository Implementations

- Customizing Individual Repositories
  - Configuration
- Customize the Base Repository
- Customizing the Repository Factory
- Customize the Repository Factory Bean
- Using JpaContext in Custom Implementations

Spring Data provides various options to create query methods with little coding. But when those options don't fit your needs you can also provide your own custom implementation for repository methods. This section describes how to do that.

Customizing Individual Repositories

To enrich a repository with custom functionality, you must first define a fragment interface and an implementation for the custom functionality, as follows:

Interface for custom repository functionality

```java
interface CustomizedUserRepository {
  void someCustomMethod(User user);
}
```

Implementation of custom repository functionality

```java
class CustomizedUserRepositoryImpl implements CustomizedUserRepository {

  @Override
  public void someCustomMethod(User user) {
    // Your custom implementation
  }
}
```

NOTE

NOTE

The most important part of the class name that corresponds to the fragment interface is the `Impl` postfix. You can customize the store-specific postfix by setting `@Enable<StoreModule>Repositories(repositoryImplementationPostfix = …)`.

WARNING

Historically, Spring Data custom repository implementation discovery followed a naming pattern that derived the custom implementation class name from the repository allowing effectively a single custom implementation.

A type located in the same package as the repository interface, matching *repository interface name* followed by *implementation postfix*, is considered a custom implementation and will be treated as a custom implementation. A class following that name can lead to undesired behavior.

We consider the single-custom implementation naming deprecated and recommend not using this pattern. Instead, migrate to a fragment-based programming model.

The implementation itself does not depend on Spring Data and can be a regular Spring bean. Consequently, you can use standard dependency injection behavior to inject references to other beans (such as a `JdbcTemplate`), take part in aspects, and so on.

Then you can let your repository interface extend the fragment interface, as follows:

Changes to your repository interface

JAVA

```
interface UserRepository extends CrudRepository<User, Long>, CustomizedUserRepository {

  // Declare query methods here
}
```

Extending the fragment interface with your repository interface combines the CRUD and custom functionality and makes it available to clients.

Spring Data repositories are implemented by using fragments that form a repository composition. Fragments are the base repository, functional aspects (such as Querydsl), and custom interfaces along with their implementations. Each time you add an interface to your repository interface, you enhance the composition by adding a fragment. The base repository and repository aspect implementations are provided by each Spring Data module.

The following example shows custom interfaces and their implementations:

Fragments with their implementations

JAVA

```
interface HumanRepository {
  void someHumanMethod(User user);
}
```

```
class HumanRepositoryImpl implements HumanRepository {

  @Override
  public void someHumanMethod(User user) {
    // Your custom implementation
  }
}

interface ContactRepository {

  void someContactMethod(User user);

  User anotherContactMethod(User user);
}

class ContactRepositoryImpl implements ContactRepository {

  @Override
  public void someContactMethod(User user) {
    // Your custom implementation
  }

  @Override
  public User anotherContactMethod(User user) {
    // Your custom implementation
  }
}
```

The following example shows the interface for a custom repository that extends `CrudRepository`:

Changes to your repository interface

JAVA

```
interface UserRepository extends CrudRepository<User, Long>, HumanRepository,
ContactRepository {

  // Declare query methods here
}
```

Repositories may be composed of multiple custom implementations that are imported in the order of their declaration. Custom implementations have a higher priority than the base implementation and repository aspects. This ordering lets you override base repository and aspect methods and resolves ambiguity if two fragments contribute the same method signature. Repository fragments are not limited to use in a single repository interface. Multiple repositories may use a fragment interface, letting you re-use customizations across different repositories.

The following example shows a repository fragment and its implementation:

Fragments overriding save(…)

```java
interface CustomizedSave<T> {
  <S extends T> S save(S entity);
}

class CustomizedSaveImpl<T> implements CustomizedSave<T> {

  @Override
  public <S extends T> S save(S entity) {
    // Your custom implementation
  }
}
```

The following example shows a repository that uses the preceding repository fragment:

Customized repository interfaces

```java
interface UserRepository extends CrudRepository<User, Long>, CustomizedSave<User> {
}

interface PersonRepository extends CrudRepository<Person, Long>, CustomizedSave<Person> {
}
```

Configuration

The repository infrastructure tries to autodetect custom implementation fragments by scanning for classes below the package in which it found a repository. These classes need to follow the naming convention of appending a postfix defaulting to `Impl`.

The following example shows a repository that uses the default postfix and a repository that sets a custom value for the postfix:

Example 1. Configuration example

Java | **XML**

```java
@EnableJpaRepositories(repositoryImplementationPostfix = "MyPostfix")
class Configuration { … }
```

The first configuration in the preceding example tries to look up a class called `com.acme.repository.CustomizedUserRepositoryImpl` to act as a custom repository implementation. The second example tries to look up `com.acme.repository.CustomizedUserRepositoryMyPostfix`.

Resolution of Ambiguity

If multiple implementations with matching class names are found in different packages, Spring Data uses the bean names to identify which one to use.

Given the following two custom implementations for the `CustomizedUserRepository` shown earlier, the first implementation is used. Its bean name is `customizedUserRepositoryImpl`, which matches that of the fragment interface (`CustomizedUserRepository`) plus the postfix `Impl`.

Example 2. Resolution of ambiguous implementations

```java
class CustomizedUserRepositoryImpl implements CustomizedUserRepository {

  // Your custom implementation
}
```

```java
@Component("specialCustomImpl")
class CustomizedUserRepositoryImpl implements CustomizedUserRepository {

  // Your custom implementation
}
```

If you annotate the `UserRepository` interface with `@Component("specialCustom")`, the bean name plus `Impl` then matches the one defined for the repository implementation in `com.acme.impl.two`, and it is used instead of the first one.

Manual Wiring

If your custom implementation uses annotation-based configuration and autowiring only, the preceding approach shown works well, because it is treated as any other Spring bean. If your implementation fragment bean needs special wiring, you can declare the bean and name it according to the conventions described in the preceding section. The infrastructure then refers to the manually defined bean definition by name instead of creating one itself. The following example shows how to manually wire a custom implementation:

Example 3. Manual wiring of custom implementations

Java | **XML**

```java
class MyClass {
  MyClass(@Qualifier("userRepositoryImpl") UserRepository userRepository) {
    …
```

```
    }
}
```

Registering Fragments with spring.factories

As already mentioned in the Configuration section, the infrastructure only auto-detects fragments within the repository base-package. Therefore, fragments residing in another location or want to be contributed by an external archive will not be found if they do not share a common namespace. Registering fragments within `spring.factories` allows you to circumvent this restriction as explained in the following section.

Imagine you'd like to provide some custom search functionality usable across multiple repositories for your organization leveraging a text search index.

First all you need is the fragment interface. Note the generic `<T>` parameter to align the fragment with the repository domain type.

Fragment Interface

```java
public interface SearchExtension<T> {

    List<T> search(String text, Limit limit);
}
```

Let's assume the actual full-text search is available via a `SearchService` that is registered as a `Bean` within the context so you can consume it in our `SearchExtension` implementation. All you need to run the search is the collection (or index) name and an object mapper that converts the search results into actual domain objects as sketched out below.

Fragment implementation

```java
class DefaultSearchExtension<T> implements SearchExtension<T> {

    private final SearchService service;

    DefaultSearchExtension(SearchService service) {
        this.service = service;
    }

    @Override
    public List<T> search(String text, Limit limit) {
        return search(RepositoryMethodContext.getContext(), text, limit);
    }
```

```
    List<T> search(RepositoryMethodContext metadata, String text, Limit limit) {

        Class<T> domainType = metadata.getRepository().getDomainType();

        String indexName = domainType.getSimpleName().toLowerCase();
        List<String> jsonResult = service.search(indexName, text, 0, limit.max());

        return jsonResult.stream().map(…).collect(toList());
    }
}
```

In the example above `RepositoryMethodContext.getContext()` is used to retrieve metadata for the actual method invocation. `RepositoryMethodContext` exposes information attached to the repository such as the domain type. In this case we use the repository domain type to identify the name of the index to be searched.

Exposing invocation metadata is costly, hence it is disabled by default. To access `RepositoryMethodContext.getContext()` you need to advise the repository factory responsible for creating the actual repository to expose method metadata.

Expose Repository Metadata

Marker Interface	**Bean Post Processor**

Adding the `RepositoryMetadataAccess` marker interface to the fragments implementation will trigger the infrastructure and enable metadata exposure for those repositories using the fragment.

JAVA
```
class DefaultSearchExtension<T> implements SearchExtension<T>, RepositoryMetadataAccess
{

    // ...
}
```

Having both, the fragment declaration and implementation in place you can register the extension in the `META-INF/spring.factories` file and package things up if needed.

Register the fragment in `META-INF/spring.factories`

PROPERTIES
```
com.acme.search.SearchExtension=com.acme.search.DefaultSearchExtension
```

Now you are ready to make use of your extension; Simply add the interface to your repository.

Using it

JAVA
```
```

```
interface MovieRepository extends CrudRepository<Movie, String>, SearchExtension<Movie> {

}
```

Customize the Base Repository

The approach described in the preceding section requires customization of each repository interfaces when you want to customize the base repository behavior so that all repositories are affected. To instead change behavior for all repositories, you can create an implementation that extends the persistence technology-specific repository base class. This class then acts as a custom base class for the repository proxies, as shown in the following example:

Custom repository base class

JAVA

```
class MyRepositoryImpl<T, ID>
  extends SimpleJpaRepository<T, ID> {

  private final EntityManager entityManager;

  MyRepositoryImpl(JpaEntityInformation entityInformation,
                           EntityManager entityManager) {
    super(entityInformation, entityManager);

    // Keep the EntityManager around to used from the newly introduced methods.
    this.entityManager = entityManager;
  }

  @Override
  @Transactional
  public <S extends T> S save(S entity) {
    // implementation goes here
  }
}
```

CAUTION

The class needs to have a constructor of the super class which the store-specific repository factory implementation uses. If the repository base class has multiple constructors, override the one taking an `EntityInformation` plus a store specific infrastructure object (such as an `EntityManager` or a template class).

The final step is to make the Spring Data infrastructure aware of the customized repository base class. In configuration, you can do so by using the `repositoryBaseClass`, as shown in the following example:

Example 4. Configuring a custom repository base class

Java | XML

```java
@Configuration
@EnableJpaRepositories(repositoryBaseClass = MyRepositoryImpl.class)
class ApplicationConfiguration { … }
```

Customizing the Repository Factory

Customizing the repository factory through `RepositoryFactoryCustomizer` provides direct access to components involved with repository instance creation. This mechanism is useful when you want to adjust selected aspects of proxy creation without introducing a fully custom repository factory bean. The following example, demonstrates registering additional listeners and proxy advisors:

```java
factoryBean.addRepositoryFactoryCustomizer(repositoryFactory -> {
    repositoryFactory.addInvocationListener(…);
    repositoryFactory.addQueryCreationListener(…);

    repositoryFactory.addRepositoryProxyPostProcessor((factory, repositoryInformation) ->
factory.addAdvice(…)));
});
```

A `RepositoryFactoryCustomizer` is associated with a particular repository factory bean, ideally through `BeanPostProcessor` so that only specific repositories are affected. Note that customizer beans are not applied automatically to prevent unwanted wiring that become especially relevant in multi-repository arrangements or when using multiple Spring Data modules.

Customize the Repository Factory Bean

The most powerful approach to customize repository creation is to provide a custom repository factory bean, typically a subclass of `RepositoryFactorySupport`, `TransactionalRepositoryFactoryBeanSupport` or the store-specific repository factory bean.

Customizing the repository factory bean allows you to change repository creation entirely with full access to the underlying repository factory.

Note that this approach requires the most effort and is typically only needed when you want to change core aspects of repository creation. Also, you need to take in consideration repository metadata derivation that is used to identify query methods, base implementation methods and custom implementations. The following summary outlines the key aspects:

- `repositoryBaseClass` : The repository base class defines which methods are implemented by the base class and which methods require additional handling through aspects or custom implementations.
- `repositoryFragmentsContributor` : A `RepositoryFragmentsContributor` allows contributions to repository composition after all standard fragments have been collected. Store modules use this mechanism to add features such as Querydsl or Query-by-Example support. It also serves as an SPI for third-party extensions.
- `exposeMetadata` : Controls whether invocation metadata is available through `RepositoryMethodContext.getContext()` .

Using `JpaContext` in Custom Implementations

When working with multiple `EntityManager` instances and custom repository implementations, you need to wire the correct `EntityManager` into the repository implementation class. You can do so by explicitly naming the `EntityManager` in the `@PersistenceContext` annotation or, if the `EntityManager` is `@Autowired` , by using `@Qualifier` .

As of Spring Data JPA 1.9, Spring Data JPA includes a class called `JpaContext` that lets you obtain the `EntityManager` by managed domain class, assuming it is managed by only one of the `EntityManager` instances in the application. The following example shows how to use `JpaContext` in a custom repository:

Example 5. Using `JpaContext` in a custom repository implementation

JAVA

```
class UserRepositoryImpl implements UserRepositoryCustom {

  private final EntityManager em;

  @Autowired
  public UserRepositoryImpl(JpaContext context) {
    this.em = context.getEntityManagerByManagedType(User.class);
  }

  …
}
```

The advantage of this approach is that, if the domain type gets assigned to a different persistence unit, the repository does not have to be touched to alter the reference to the persistence unit.